NNDL project report template

Edoardo Furlan†, Andrea Marigo†, Francesca Zorzi†

***Abstract*—In the field of generative AI, the task of music generation is heavily conditioned by the need of training a network that not only provides notes, but is able to produce pieces of music made of a coherent sequence of notes, and may be good for the human ear. The network we developed is a Generative Adversarial Network (GAN) inspired by MidiNet (qua va aggiunta la reference) that is modified by including a Contitioner network inside the Generator (CGAN) and a Gated Recurrent Unit (GRU). The Conditioner uses features extracted from previously generated bars and feeds them to the Generator at each layer, providing a guide to maintain musical context. The Recurrent Unit is able to maintain an hidden state that, combined with the features extracted by the Conditioner, is able to maintain a theme of the piece of music that is being generated. This theme is combined with the Conditioner's features at the beginning of the generation process, forcing the network to maintain coherent melodies.**

The result is an integration of recurrent memory and convolutional layers that leads to improved thematic coherence. While limited to short piano music, improvements on this architecture could lead to the generation of more complex and multi-instrumental compositions

I suggest to write the Abstract as the very last thing. You may sketch it at the beginning, but then always finalize it at the end.

***Index Terms*—Neural Networks, Convolutional Neural Networks, Generative Adversarial Networks, Symbolic-Domain Music, Gated Recurrent Unit. A list of keywords defining the tools and the scenario. I would not go beyond *six* keywords.**

I. Introduction

Remark I.1. Short Intro: The task of generating music is a frontier in Artificial Intelligence, since it poses the challenge of training a network to be creative, but with the rigid constraints imposed by music theory. While some architectures are already capable of generating technically corect sequences, capturing the key points of realistic, human-like compsitions, remains an interesting topic. This is a praticularly interesting problem if seen in the scenario of creating assistive tools for composers: both experienced and non experienced ones would benefit by these tools in order to improve creativity and exploration of genres. What we propose is a hybrid architecture that aims no only to model the placement of notes, but also their intensity and the evolution of the themaic they represent.

Remark I.2. Paper contribution: In music generation, coherence is the main objective. That's why a simple RNN architecture seems to be the most tappropriate solution, as it's structure aims at learning sequences in order to predict the next values. However, MidiNet showed that convolutional GANs can overcome the issue of learning time related forgetting by using spatial conditioning, while achieving better performances regarding creativity, since the outputs directly come from random noise, thus allowing the network to generate new music from scratch. The network that we propose tries to combine these two point of view, by adding to the generation process a recurrent unit that will help in learning long-term relations. Additionally, we used a dataset that takes into account velocity (intensity of the notes). This means that the patterns that the network has to learn become much more complex and may not be captured if the network is not suitably structured to handle the new relations between notes. Another relevant change with respect to the original MidiNet archtecture lies in the Discriminator network. In its input we propose the use of both the current and the previous bars from the dataset, in order to include in the discrimination process a similar approach to the one regarding the conditioner in the generator. This changes aim to improve the quality of the game played by Generator and Discriminator, which should eventually lead to a more realistic result.

Remark I.3. Summary of contributions:

- Hybrid Recurrent-Convolutional Network: We extended the convolutional MidiNet with a Gated Recurrent unit within the generator
- Multi-Dimensional Conditioning: The model includes a dual-conditioning mechanism, which combines local features via the Discriminator and global thematic via the GRU
- Modelling Velocity: Our network is built to include note only note placement, but also to choose their intensity. The goal is to improve the realness of the produced pieces.
- Transition-aware Discriminator: We propose an improved adversarial game with an enhanced Discriminator that evaluates subsequent bars and chord embeddings.

Remark I.4. How to organize your writeup with Latex: I find it useful, especially for a paper with multiple authors, to split your manuscript into multiple source Latex files. This is very easily accomplished by having single *root* file, e.g., `main.tex`, where you define the article class, all the user-defined commands, the paper formatting options (e.g., one vs two columns, the page margins, the text size and the fonts, etc.). For this project, the root is called `template.tex`. From the root you then call, in sequential order, a number of other latex source files through

†Department of Information Engineering, University of Padova, email: {edoardo.furlan.3, andrea.marigo.3, francesca.zorzi.3}@studenti.unipd.it

the command `\input{filename.tex}`. It is often convenient to have one of such files for each section of your paper. This facilitates editing multiple sections in parallel and keeping your project synchronized with some versioning and revision control system such as or `git` or `overleaf` (highly recommended).

Remark I.5. Compiling trick: For each source Latex file that you call from the main, I recommend you copy the following command in the very first line

```
% !TEX root = template.tex
```

where `template.tex` is the name of the root Latex file. This command tells the Latex compiler that your main root file is `template.tex`, and it makes it possible to compile the paper from any of the sub-files. Example: imagine you are editing `introduction.tex` and have that open in a window of your preferred Latex editor. With this command, you can compile from the local `introduction.tex` window, without having to open and switch every time to the root file window and compile from it. This will save you hours.

Maximum length for the whole report is 6 pages. Abstract, introduction and related work should take max 1 or 1.5 pages.

Recommended structure for the intro: you may use the following structure.

- **Present the paper contribution:** A third paragraph were you state what you do in the paper, this should also be concisely written. A good rule of thumb is to make it max 10/15 lines. Here, you should state up front:
  1) **problem**: the problem at stake,
  2) **relevance**: the relevance and timeliness of what you propose,
  3) **approach**: the technique/approach you use, possibly underlying its novelty, efficiency,
  4) **value**: underline the value/novelty of your proposal referencing (recent) papers from the literature,
  5) **applicability**: tell the reader how she/he can take advantage of your work, e.g., how your work/results can be reused/exploited to achieve further scientific, technical or practical (integrated into products?) goals.
- **Summary of contributions:** After this, you may want to provide an itemized list to summarize the paper contributions. Rule of thumb: from three to six items, from three to four lines each.
- **Closing (paper structure):** You finish up by detailing the paper structure, this should be three to four lines. It is customary to do so, although I admit it may be of little use. It usually goes like: *"This report (paper) is structured as follows. In Section II we describe the state of the art, the system and data models are respectively presented in Sections III and IV. The proposed signal processing technique is detailed in Section V and its performance evaluation is carried out in Section VI. Concluding remarks are provided in Section VII."*

Remark I.6. Lately, I tend to write introduction plus abstract within a single page. This forces me to focus on the important messages that I want to deliver about the paper, leaving out all the "blah blah". **Remember:** 1) *less is more*, 2) writing a compact (*snappy*) piece of technical text is much more difficult than writing lengthy stuff with no space constraints.

II. Related Work

Some hints:

- **Goal:** The goal of this section is to describe what has been done so far in *the* literature. You should focus on and briefly describe the work done in the best papers that you have read.
- **Length:** One full column is fine but often this takes one column and a half. It is very easy to use a full page, although this may just due to your sloppiness... if you carefully go through the one page long version, you often find it possible to compact it in one column and a half. In any event, I would make this section no longer than one page, this leads to an overall *two pages* including abstract, introduction and related work. I believe this is a fair amount of space in most cases.
- **Approach:** For each you should comment on the paper's contribution, on the good and important findings of such paper and also, **1)** on why these findings are not enough and **2)** how these findings are improved upon/extended by the work that you present here. At the end of the section, you may recap the main paper contributions (maybe one or two, the most important ones) and how these extend/improve upon previous work.
- **References:** please follow this *religiously*. It will help you a lot. Use the Latex `Bibtex` tool to manage the bibliography. A `Bibtex` example file, named `biblio.bib` is provided with this template.

- **Citing conference/workshop papers:** I recommend to always include the following information into the corresponding `bibitem` entry:
  1) author names,
  2) paper title,
  3) conference / workshop name,
  4) conference / workshop address,
  5) month,
  6) year.

  Examples of this are: [1] [2].

- **Citing journal papers:** I recommend to always include the following information into the corresponding `bibitem` entry:
  1) author names,
  2) paper title,
  3) full journal name,
  4) volume (if available),
  5) number (if available),

6) month,
7) pages (if available),
8) year.

Examples of this are: [3] [4] [5].

- **Citing books:** I recommend to always include the following information into the corresponding `bibitem` entry:
  1) author names,
  2) book title,
  3) editor,
  4) edition,
  5) year.

Remark II.1. Note that some of the above fields may not be shown when you compile the Latex file, but this depends on the bibliography settings (dictated by the specific Latex style that you load at the beginning of the document). You may decide to include additional pieces of information in a given bibliographic entry, but please, **be consistent** across all the entries, i.e., use the same fields for the same publication type. Note that some of the fields may not be available (e.g., the paper *volume*, *number* or the *pages*).

III. Processing Pipeline

Remark III.1. On tailoring the paper structure to your needs: The structure recommended for the previous sections is rather standard and could work for different papers with differing technical content, the structure and the paper content from here on highly depends on the type of paper, possibilities are: mostly based on theoretical analysis, showing experimental design/activity, proposing a new technique and analyzing its performance via experiments or simulation. For the HDA course we deal with machine learning and, in detail, with training and testing neural network architectures to perform some specific inference or classification task. The following structure and comments are specifically addressing this type of technical content.

Remark III.2. Why having this section: With this section, we start the technical description with a *high level* introduction of your work (e.g., processing pipeline). Here, you do not have to necessarily go into the technical details of every block/algorithm of your design, this will be done later as the paper develops. What I would like to see here is a high level description of the approach, i.e., which processing blocks you used, what they do (in words) and how these were combined, etc. This section should introduce the reader to your design, explain the different parts/blocks, how they interact and why. You should not delve into technical details for each block, but you should rather explain the big picture. Besides a well written explanation, I often use a nice diagram containing the various blocks and detailing their interrelations.

Writing tips: Sections, Figures and Tables are usually shortened as Sec., Fig., Tab.

- **Cross referencing:** In Latex, cross referencing is easy. You need to label an object through the `\label{labelid}` command and referencing it where you need it through the `\ref{labelid}` command.
- **Suggestion:** I suggest to cross reference a table using `Tab.~\ref{tab:tableid}`, the same holds for figures and sections, by just replacing "`Tab.`" with "`Fig.`" and "`Sec.`". Of course when defining the table, you need to add a Latex command `\label{tab:tableid}` in the right place inside the table Latex environment to cross-link it. The tag `tab:tableid` is user defined and is the identifier that you associate with the table in question. You could call it `pippo` of you wish, but I recommend to use something like "`tab:tableid`" for tables, "`eq:eqid`" for equations, "`sec:secid`" for sections and so forth, where `tableid` has to be unique for each table in the document. The same applies to figures and all other objects. I guess you got the idea. This will lead to a neat Latex code and will facilitate cross referencing while avoiding duplicate labels, especially in large Latex documents (think of a book for example).
- **But what about the tilde?** When you write `Tab.~\ref{tab:tableid}` Latex knows that `Tab.` and the corresponding table number `\ref{tab:tableid}` must be displayed within the same line, i.e., they can never be broken across lines. This is nice and desirable I believe. I always use it for all referenced material, including citations; example: "As done in`~\cite{suppa-wu-2019}`."
- **More about breaking stuff across lines** often times you have composed words, in line equations, etc. and for some reason you would like Latex to never break them across lines. Example: the Latex command `\mbox{Neural Networks (NN)}` is processed by Latex so that "Neural Networks (NN)" is never broken across lines. I use this very often, also for inline equations that I do not want Latex to split across subsequent lines.

IV. Signals and Features

Being a machine learning paper, I would have here a section describing the signals you have been working on. If possible, you should describe, in order,

1) the measurement setup (if relevant), how input data is formatted (e.g., vectors of fixed size measured at regular sampling times),
2) how the signals were pre-processed, e.g., to remove noise, artifacts, fill gaps (missing points) or to re-interpolate the signals to represent them through a constant sampling rate, etc.
3) after this, you should describe how *feature vectors* were obtained from the pre-processed signals. If signals are *time series* this also implies stating the segmentation / windowing strategy that you adopted, to then describe how you obtained a feature vector for each time window. Also, if you use existing feature extraction approaches, you may want to briefly describe them as well, in

addition to (and before) your own (possibly new) feature extraction method.

Last but not least, this section should also contain information on how you have split the dataset into training, validation and test sets. This will be briefly recalled within the "Results" section.

V. Learning Framework

Here you finally describe the learning strategy / algorithm that you conceived and used to solve the problem at stake. A good diagram to exemplify how learning is carried out is often very useful. In this section, you should describe the learning model, its parameters, any optimization over a given parameter set, etc. You can organize this section into sub-sections. You are free to choose the most appropriate structure.

Remark V.1. Note that the diagram that you put here differs from that of Section III as here you show the details of how your learning framework, or the core of it, is built. In Section III you instead provide a high-level description of the involved processing blocks, i.e., you describe the *processing flow* and the rationale behind it.

On math typesetting: there are many Latex tricks that you should use to produce a high quality technical essay. A few are listed below, in random order:

- **Vectors and matrices:** x is a scalar, whereas $\boldsymbol{x}$ (in bold) is a vector, and $\boldsymbol{X}$ is a matrix with elements $\boldsymbol{X} = [x_{ij}]$. For bold symbols you may use the `\bm` Latex command, e.g., `\bm(x)`.
- **Operators:** such as max, min, argmax, argmin and special functions such as $\log(\cdot)$, $\exp(\cdot)$, $\sin(\cdot)$, $\cos(\cdot)$ are obtained through specific latex commands `\min`, `\max`, `\arg\!min`, `\arg\!max`, `\log`, `\exp`, `\sin`, `\cos`, etc. Use them! log, exp, min, sin, cos, etc., look ugly.
- **Sets** can be represented through calligraphic fonts, e.g., $\mathcal{S}$, $\mathcal{F}$, $\mathcal{B}$, etc., obtained using the Latex command `\mathcal{S}`, etc.
- **Equations:** for a single equation use the `equation` Latex environment. Example:

$$\sigma(x) = \frac{1}{1+e^{-x}}. \tag{1}$$

Now, using round brackets (and) we get

$$\sigma(x) = (\frac{1}{1+e^{-x}}), \tag{2}$$

but this looks ugly, you should use "`\left (`" for "(" and "`\right )`" for ")", obtaining

$$\sigma(x) = \left(\frac{1}{1+e^{-x}}\right). \tag{3}$$

- **Punctuation:** Displayed equations are usually considered to be part of the text and, in turn, they will get the very same punctuation as if they were inline with the text (and part of the sentence). If the sentence ends with a displayed equation, the equation gets a period "." right after it, see Eq. (1). iI the equation is instead part of a running sentence, which is continued after it, then the equation may be ended by a "," as in Eq. (2). Use the standard grammar rules and your good sense of flow to assess how equations should be punctuated, I usually read through as if they were plain text.

VI. Results

In this section, you should provide the numerical results. You are free to decide the structure of this section. As a general "rule of thumb", use plots to describe your results, showing, e.g., precision, recall and F-measure as a function of the system (learning) parameters. You can also show the precision matrix.

Remark VI.1. Present the material in a progressive and logical manner, starting with simple things and adding details and explaining more complex findings as you go. Also, do not try to explain/show multiple concepts within the same sentence. Try to **address one concept at a time**, explain it properly, and only then move on to the next one.

Remark VI.2. The best results are obtained by generating the graphs using a vector type file, commonly, either `encapsulated postscript (eps)` or `pdf` formats. To plot your figures, use the Latex `\includegraphics` command. Lately, I tend to use pdf more.

Remark VI.3. If your model has hyper-parameters, show selected results for several values of these. Usually, tables are a good approach to concisely visualize the performance as hyper-parameters change. It is also good to show the results for different flavors of the learning architecture, i.e., how architectural choices affect the overall performance. An example is the use of CNN only or CNN with adversarial training, or using residual layers for CNNs, dropout for better generalization or autoencoder models. So you may obtain different models that solve the same problem, e.g., CNN, CNN+residual layers, etc.

VII. Concluding Remarks

This section should take max half a column, I personally find it difficult to come up with really useful observations, I mean ones that bring a new contribution with respect to what you have already expounded in the "Results" section..

In many papers, here you find a summary of what done. It is basically an abstract where instead of using the present tense you use the past participle, as you refer to something that you have already developed in the previous sections. While I did it myself in the past, I now find it rather useless.

What I would like to see here is:

1) a very short summary of what done,
2) some (possibly) intelligent observations on the relevance and *applicability* of your algorithms / findings,

3) what is still missing, and can be added in the future to extend your work.

The idea is that this section should be *useful* and not just a repetition of the abstract (just re-phrased and written using a different tense...).

Moreover: being a project report, I would also like to see a specific paragraph stating

4) what you have learned, and
5) any difficulties you may have encountered.

VIII. Exam rules

What you need to do to pass the exam:

- Optional: team up with other students. Max. group size is **three students** per group;
- Identify a project to work on, devise your own neural network architecture and test it on the provided dataset;
- **Pass the written exam:** You can start working on the project before passing the written exam but you can only submit the report and code after passing it.
- **Prepare a written project report:** Follow the template above to be sure to include all the required content;
- **Submit the project for evaluation:** Submit i) your written report and ii) the code *as two separate files*;

Your final project score will be obtained taking into account the following criteria:

- **Project** (70 points): originality (10 pt.) - data preprocessing techniques (10 pt.) - learning architectures (20 pt.) - comparison against baseline/other approaches (20 pt.)
- **Written report** (30 points): clarity (10 pt.) - completeness (10 pt.) - analysis of results (number and type of metrics used) (10 pt.)

The score on the project is computed as

$$\text{Project score} = \frac{\text{tot\_points} \times 30}{90}. \tag{4}$$

The final grade will be computed as the average of the score in the written exam (50%) and that on the project (50%).

References

[1] M. Zargham, A. Ribeiro, A. Ozdaglar, and A. Jadbabaie, "Accelerated dual descent for network optimization," in *American Control Conference*, (San Francisco, CA, US), June 2011.

[2] C. M. Sadler and M. Martonosi, "Data compression algorithms for energy-constrained devices in delay tolerant networks," in *ACM SenSys*, (Boulder, CO, US), Oct. 2006.

[3] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, pp. 379–423, July 1948.

[4] S. Boyd, N. Parikh, E. Chu, and B. P. J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," *Foundations and Trends in Machine Learning*, vol. 3, pp. 1–122, Jan. 2011.

[5] D. Zordan, B. Martinez, I. Vilajosana, and M. Rossi, "On the Performance of Lossy Compression Schemes for Energy Constrained Sensor Networking," *ACM Transactions on Sensor Networks*, vol. 11, pp. 15:1–15:34, Aug. 2014.